CSC240_BST/QueType.cpp

```
#include <cstddef>                // For NULL
#include <new>                    // For bad_alloc
#include "QueType.h"

struct NodeType
{
  ItemType info;
  NodeType* next;
};



QueType::QueType()          // Class constructor.
// Post:  front and rear are set to NULL.
{
  front = NULL;
  rear = NULL;
}


void QueType::MakeEmpty()
// Post: Queue is empty; all elements have been deallocated.
{
  NodeType* tempPtr;

    while (front != NULL)
  {
    tempPtr = front;
    front = front->next;
    delete tempPtr;
  }
  rear = NULL;
}

          // Class destructor.
QueType::~QueType()
{
  MakeEmpty();
}


bool QueType::IsFull() const
// Returns true if there is no room for another ItemType
//  on the free store; false otherwise.
{
  NodeType* location;
  try
```

```
48    {
49      location = new NodeType;
50      delete location;
51      return false;
52    }
53    catch(std::bad_alloc&)
54    {
55      return true;
56    }
57  }
58
59
60  bool QueType::IsEmpty() const
61  // Returns true if there are no elements on the queue; false otherwise.
62  {
63    return (front == NULL);
64  }
65
66
67  void QueType::Enqueue(ItemType newItem)
68  // Adds newItem to the rear of the queue.
69  // Pre:  Queue has been initialized.
70  // Post: If (queue is not full) newItem is at the rear of the queue;
71  //       otherwise a FullQueue exception is thrown.
72
73  {
74    if (IsFull())
75      throw FullQueue();
76    else
77    {
78      NodeType* newNode;
79
80      newNode = new NodeType;
81      newNode->info = newItem;
82      newNode->next = NULL;
83      if (rear == NULL)
84        front = newNode;
85      else
86        rear->next = newNode;
87      rear = newNode;
88    }
89  }
90
91
92  void QueType::Dequeue(ItemType& item)
93  // Removes front item from the queue and returns it in item.
94  // Pre:  Queue has been initialized and is not empty.
95  // Post: If (queue is not empty) the front of the queue has been
96  //       removed and a copy returned in item;
```

```
97  //       othersiwe a EmptyQueue exception has been thrown.
98  {
99    if (IsEmpty())
100     throw EmptyQueue();
101   else
102   {
103     NodeType* tempPtr;
104
105     tempPtr = front;
106     item = front->info;
107     front = front->next;
108     if (front == NULL)
109       rear = NULL;
110     delete tempPtr;
111   }
112 }
113
114 QueType::QueType
115   (const QueType& anotherQue)
116 {
117   NodeType* ptr1;
118   NodeType* ptr2;
119
120   if (anotherQue.front == NULL)
121     front = NULL;
122   else
123   {
124     front = new NodeType;
125     front->info = anotherQue.front->info;
126     ptr1 = anotherQue.front->next;
127     ptr2 = front;
128     while (ptr1 != NULL)
129     {
130       ptr2->next = new NodeType;
131       ptr2 = ptr2->next;
132       ptr2->info = ptr1->info;
133       ptr1 = ptr1->next;
134     }
135     ptr2->next = NULL;
136     rear = ptr2;
137   }
138 }
139
```